

Lab 10: File Operations

Computer Programming (010153002) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Reading and writing text files with FILE *

Objectives

- Open and close text files with `fopen/fclose`; check for failure.
- Read with `fscanf`, `fgets`, and `fgetc`; write with `fprintf`, `fputs`, `fputc`.
- Detect end-of-file correctly using the function return value, not `feof`.
- Combine file I/O with previous skills (loops, arrays, structs) to process data sets.

Quick Reference

A file handle is a pointer. `fopen` returns a `FILE *` – a pointer to an internal bookkeeping struct managed by the standard library. You never dereference it yourself; you just pass it to other `f*` functions. If `fopen` fails it returns `NULL` – *always* check.

Open / close.

```
FILE *f = fopen("input.txt", "r");
if (f == NULL) { perror("fopen"); return 1; }
/* ... use f ... */
fclose(f);
```

Modes. "r" read, "w" write (truncates), "a" append, add "+" for read+write.

Read. `fscanf(f, "%d", &x)` returns the number of items successfully read, or EOF.
`fgets(line, sizeof line, f)` returns `NULL` at end of file or on error.

Idiomatic read loop.

```
int x;
while (fscanf(f, "%d", &x) == 1) {
    /* process x */
}
```

Write. `fprintf(f, "%d %s\n", id, name);`

Quick Experiments (Try It)

Try It 1: What does `fopen` return for a missing file?

Run and observe:

```
#include <stdio.h>
int main(void) {
    FILE *f = fopen("no_such_file.txt", "r");
    if (f == NULL)
        printf("fopen returned NULL -- file not found\n");
    else {
        printf("fopen succeeded (unexpected!)\n");
        fclose(f);
    }
    return 0;
}
```

1. Change "r" to "w". Does the file now get created? Verify in your directory.
2. Change "w" to "a" and run twice. Does the file grow?
3. Why is checking for NULL after `fopen` non-optional?

Try It 2: "w" truncates; "a" appends

Create a file `test.txt` (any content). Then run this program *twice* and inspect `test.txt` after each run:

```
#include <stdio.h>
int main(void) {
    FILE *f = fopen("test.txt", "w");    /* try "a" on the second run */
    if (!f) { perror("fopen"); return 1; }
    fprintf(f, "run result\n");
    fclose(f);
    return 0;
}
```

1. With mode "w": how many lines does `test.txt` have after the second run? Did the first run's content survive?
2. Change to "a" and run three times. How many lines does the file have now?
3. Which mode would you use for a log file that must never lose old entries?

Try It 3: `fprintf(stdout, ...)` is the same as `printf(...)`

Run and redirect the output to a file from your terminal:

```
#include <stdio.h>
int main(void) {
    printf("1: from printf\n");
    fprintf(stdout, "2: from fprintf(stdout)\n");
    fprintf(stderr, "3: from fprintf(stderr)\n");
    return 0;
}
```

1. Run normally. Which lines appear on the terminal?
2. Run as `./program > out.txt`. Open `out.txt`. Which lines are in the file? Which still appear on the terminal?
3. What does this tell you about the difference between `stdout` and `stderr` – and why is separating them useful in a real program?

In-Lab Exercises (target: 2 hours)

Before you start: create a sample `numbers.txt` (one integer per line) and a `lines.txt` (any text) in your working directory.

In-Lab Exercise 1: Copy a Text File (25 min)

Write a program that reads `lines.txt` line-by-line with `fgets` and writes each line to `copy.txt`. Verify by opening `copy.txt` in your editor. Handle the case where the input file cannot be opened.

In-Lab Exercise 2: Sum and Average from File (30 min)

Read all integers from `numbers.txt` (unknown count) with `fscanf`. Print the count, sum, and average. Use the return value of `fscanf` to detect end of file – do not call `feof` as the loop condition.

In-Lab Exercise 3: Word and Line Count (35 min)

Read `lines.txt` character by character with `fgetc`. Count and print: the number of characters, the number of words (whitespace-separated), and the number of lines.

In-Lab Exercise 4: Students Report (30 min)

Create `students.txt` with lines of the form `<id> <name> <score>`, e.g. `1023 Bob 87.5`. Read the records into an array of structs (reuse the `Student` struct from Lab 8) and write `report.txt` with the same records sorted by score in descending order.

AI Collaboration**Try these prompts with an AI tool:**

- “Explain why checking `fopen`’s return value for `NULL` is non-optional, and what `perror` prints when it fails.”
- “What is the `feof`-based loop anti-pattern in C, and why does `while (fscanf(f, \"%d\", &x) == 1) fix it?`”
- “What happens to a file’s existing contents when I open it with mode `\"w\"` versus `\"a\"`?”

Critical check – verify, don’t just accept: Ask the AI to write a complete program that reads integers from a file and prints their average. Before running, inspect the code for: (a) `NULL` check after `fopen`, (b) loop termination based on `fscanf`’s return value, (c) a `fclose` call. Then test with a file containing one integer, and again with an empty file. Record whether the AI’s code handles division-by-zero on the empty case.

Know the limit: AI tools regularly omit the `fclose` call and the `NULL` check, and almost never warn you that mode `\"w\"` silently destroys an existing file – these are silent data-loss bugs, not compile errors.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Read loop terminates correctly

The file `nums.txt` contains: `10 20 30 40`. What does this print?

```
#include <stdio.h>
int main(void) {
    FILE *f = fopen("nums.txt", "r");
    int x, n = 0, s = 0;
    while (fscanf(f, "%d", &x) == 1) { s += x; n++; }
    fclose(f);
    printf("%d %d\\n", n, s);
}
```

```
    return 0;
}
```

Trace & Predict 2: "w" truncates

If `out.txt` already contains "old", what does `out.txt` contain after this program runs?

```
#include <stdio.h>
int main(void) {
    FILE *f = fopen("out.txt", "w");
    fprintf(f, "new\n");
    fclose(f);
    return 0;
}
```

Find the Bug 1: Missing check, wrong mode

List the bugs and rewrite:

```
#include <stdio.h>
int main(void) {
    FILE *f = fopen("data.txt", "w");    /* we only want to read */
    int x;
    while (!feof(f)) {                  /* anti-pattern */
        fscanf(f, "%d", &x);
        printf("%d\n", x);
    }
    /* fclose forgotten */
    return 0;
}
```

Write Code on Paper 1: Append a line

Write a complete program that prompts the user for a single line of text and appends it (followed by a newline) to `log.txt`. Create the file if it does not exist, but never erase existing content.

Write Code on Paper 2: Merge two sorted files

Two files `a.txt` and `b.txt` each contain an ascending sequence of integers. Write a complete program that produces `merged.txt` containing the combined sequence, still in ascending order, by reading one number at a time from each input.

Draw on Paper 1: File-reading flowchart

Draw a complete flowchart for a program that: opens a file; reads integers one at a time until EOF; accumulates their sum; closes the file; and prints the sum and the count.

Your diagram must include all of these elements:

- The `fopen` call and a **null-check** branch ("file opened?").
- The read-and-test loop (`fscanf` returns 1 while a value was read).
- The `fclose` step before printing results.
- Separate output for the case where the file could not be opened.

End of the lab series. Notice how every lab from Lab 7 onward has used pointers: out-parameters (Lab 7), `char *` string traversal (Lab 8), `struct *` with `->` (Lab 9), and `FILE *` here. Keep your in-lab source files and paper-work together as a portfolio for revision before the final exam.